

Codebase Guide

This document provides a comprehensive, technical walkthrough of the GRPO Small LM reinforcement learning repository. This repository is built to scale from local debugging on CPUs/single GPUs to massively parallel hyperparameter sweeps using Modal's serverless infrastructure.

1. Directory Structure Overview

```
.
├── archive/           # Historic deployment scripts (e.g., Runpod) kept for reference
├── configs/           # YAML configuration files for pipeline and training
│   └── sweeps/        # Matrix execution configurations for hyperparameter search
├── pipeline/          # Automated orchestration for sweeps and metrics extraction
├── scripts/           # Utility bash and Python scripts (UI launching, healthchecks)
├── ui/                # Streamlit analytics and inference playground
├── compare_runs.py     # Pipeline wrapper: execution entry point for comparisons
├── eval_grpo.py        # Core evaluation script (generates metrics.json)
├── grpo_config.py      # Configuration schemas specific to GRPO/TRL
├── modal_train.py      # Modal serverless deployment and distributed runner
├── reward_fns.py       # Reinforcement learning reward functions
├── run_prelim.py       # Pipeline wrapper: execution entry point for full train+eval
├── loop
│   └── train_grpo.py   # Core Hugging Face TRL GRPOTrainer script
```

2. Core Reinforcement Learning (RL) Pipeline

The primary mechanics of the GRPO (Group Relative Policy Optimization) algorithm are contained in a few flat scripts at the root level. These scripts can be run entirely standalone without the orchestration pipeline.

- **train_grpo.py**: The heart of the training loop. It parses arguments, sets up Hugging Face Accelerate/PEFT (LoRA by default for small models), initializes the datasets, and passes `reward_fns` to the `GRPOTrainer` from the `trl` library.
- **reward_fns.py**: Contains the reward calculations that guide the policy model. For example, it might contain functions that check if a generated mathematical answer exactly matches a gold standard (`accuracy_reward`) or if the formatting follows a specific `<think> -> <answer>` pattern (`format_reward`).
- **eval_grpo.py**: Loads a pre-trained or newly-tuned model, runs batched inference against a holdout test split, and computes average metrics (accuracy, formatting adherence, average generation length). Outputs a `metrics.json` file.
- **grpo_config.py**: Helper schema and parameter parsing strictly for `train_grpo.py` args (like max steps, beta KL-penalty, generation limits).

3. Pipeline Automation (pipeline/)

While `train_grpo.py` runs a **single** model, the `pipeline` module orchestrates **experiments**. An experiment might involve taking multiple baseline models, training them with various loss configurations, and comparing the evaluations against the baselines.

- **pipeline/config.py**: Implements `PrelimConfig`, which merges CLI arguments with YAML configuration files.
- **pipeline/runner.py**: Contains the logic to execute a full loop:
 1. Evaluate Baseline Model.
 2. Run `train_grpo.py` (via subprocess, potentially using `accelerate`).
 3. Evaluate Tuned Model.
 4. Repeat for all combinations of models and loss types requested.
- **pipeline/reporter.py**: Computes deltas between the baseline and the newly tuned models, generating CSV leaderboards and Markdown summaries inside the `outputs/` directory.

***Note:** The root files `run_prelim.py` and `compare_runs.py` are thin wrappers that invoke `pipeline/runner.py` and `pipeline/reporter.py`, ensuring backwards compatibility for users accustomed to the previous flat structure.*

4. Modal Distributed Execution (`modal_train.py`)

This repository is optimized for [Modal](#).

`modal_train.py` defines:

1. **Container Image**: A Debian-based image pre-loaded with `torch`, `transformers`, `trl`, `vllm`, and `accelerate`. It mounts the local repository to `/workspace`.
2. **Persistent Volume**: Automatically mounts a volume to `/workspace/outputs` so that all checkpoints, metrics, and leaderboards persist after the ephemeral GPUs spin down.
3. **Execution Logic**: `run_sweep_job` takes a specific configuration permutation (e.g., `model="Qwen/Qwen2.5-0.5B"`, `loss_type="grpo"`) and executes `run_prelim.py` inside the container.
4. **Starmap Dispatch**: The `main` function locally parses a sweep configuration (like `configs/sweeps/pipeline_sweep_presweep.yaml`) and maps over the Cartesian product of parameters, fanning out massively parallel jobs to Modal.

Usage:

```
modal run modal_train.py
```

5. UI and Analytics (`ui/`)

To visualize the massive amount of data generated by sweeps, a Streamlit app is provided.

- **ui/app.py**: The Streamlit frontend layout. It allows users to browse experiments (directories within `outputs/`), view leaderboard tables side-by-side, and inspect specific prompt/completion rows from the evaluations.
- **ui/core.py**: The backend logic. It abstracts loading `manifest.json` and `metrics.json` files, and provides an interactive “Playground” that loads a given model bundle (using `vLLM` or Hugging Face Transformers) to perform live inference directly in the browser.

Usage:

```
./scripts/run_ui.sh
```

6. Configurations (configs/)

Configurations dictate what the pipeline does. They are YAML representations of `PrelimConfig`.

- **configs/base.yaml / configs/grpo_small.yaml**: Standard single-run configurations for standard debugging.
- **configs/sweeps/*.yaml**: Configuration matrices defining multiple models, multiple learning rates, or different loss variants. `modal_train.py` defaults to reading these.

7. Diagnostics (scripts/healthcheck.py)

Before doing major refactors or submitting heavy jobs, `python scripts/healthcheck.py` evaluates your PyTorch CUDA bindings, GPU availability, and critical library versions (`trl`, `peft`, `transformers`).